A summary of Sharding

PyTorch to JAX – reviewing the right tool for Massive Models

JC Vaught

Sep 25

ML memory

- A 17.5 billion parameter model needs ~35GB just to store its weights in standard 16-bit precision
- During training, four main components consume VRAM. For every single model parameter, you need to store:
  - The model's weights. (Parameter (P)
  - The Gradient (G): Calculated during the backward pass to know how to update the weight. (Size is equal to Parameters).
  - The Optimizer States (O): Optimizers like Adam need to store extra information. This is often 2x to 4x the size of the parameters, due to being stored in higher precision.
  - The Activations (A): Intermediate outputs from the forward pass that are temporarily stored for use in the backward pass. Size depends on batch size and sequence length.

Scaling strategies

- **Data Parallelism:** (Most Common) Same model on every GPU, but each gets a different slice of the data(increase batch size basically).
- **Tensor Parallelism:** Split individual layers or operations *within* the model across different GPUs.
- **Pipeline Parallelism:** Chain GPUs together. GPU 1 runs layers 1-8, sends output to GPU 2 which runs layers 9-16, and so on.
- **Model Parallelism (General):** The classic approach of manually placing different parts of the model on different GPUs.
- Other methods – FSDP, hybrids

PyTorch by Meta AI

- **`DistributedDataParallel` (DDP):** The industry-standard, robust, and highly optimized library for data parallelism.
- **Fully Sharded Data Parallel (FSDP):** The modern answer to massive models. Shards parameters, gradients, and optimizer states across GPUs to save memory.
- Provides the low-level tools (torch.distributed) for building custom Tensor and Pipeline parallelism, offering high control but requiring A LOT more code.

DeepSpeed by Microsoft

- **ZeRO (Zero Redundancy Optimizer):** The magic behind its memory savings. A family of optimizations that eliminate redundant copies of the model, gradients, and optimizer.
- **Offloading:** Can push training components from GPU memory to CPU RAM or even NVMe storage, enabling very large model sizes.
- **3D Parallelism Made Easy:** Provides a simple API to combine data, tensor, and pipeline strategies.

Megatron-LM by NVIDIA

- ***Mainly meant for Language models*
- **Pioneered Tensor Parallelism:** Famous for its highly efficient and innovative implementation for splitting transformer layers across GPUs.
- **Advanced Techniques:** Introduces Sequence Parallelism and Interleaved Pipelines to maximize hardware utilization and reduce idle "bubbles".
- **A Performance Blueprint:** Less of a general-purpose library and more of a research framework built to achieve maximum throughput on NVIDIA hardware.

Hugging Face Accelerate

- **Ultimate Abstraction:** Hides the complexity of distributed training. You write standard, device-agnostic PyTorch code.
- **Configure, Don't Code:** Switch between different backends (DDP, FSDP, DeepSpeed) by changing a simple configuration file, not your training script.
- **Perfect for All Skill Levels:** Empowers beginners to scale up easily and allows experts to rapidly prototype and switch between strategies.

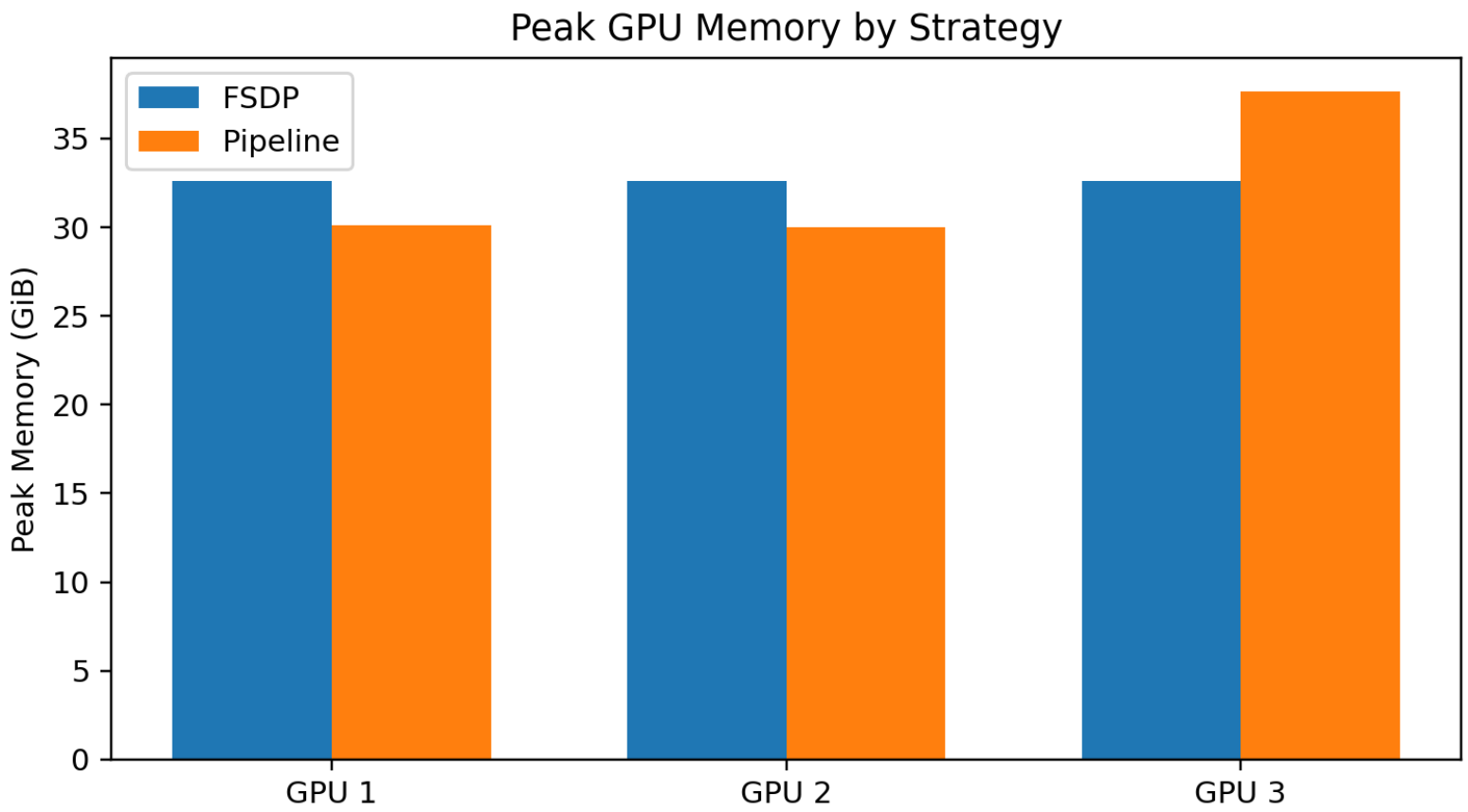
JAX by Google

- **A Different Paradigm:** JAX uses a compiler (XLA) to optimize and run Python numerical code directly on accelerators (GPUs/TPUs).
- **Parallelism as a Function:** Express parallelism explicitly with powerful functions like pmap (parallel map) and shard_map.
- **Total Control:** Define custom "device meshes" and precisely control how every single tensor is sharded across your hardware.
- This is the premier environment for getting top performance from Google's TPUs, but also exceptional for GPUs.

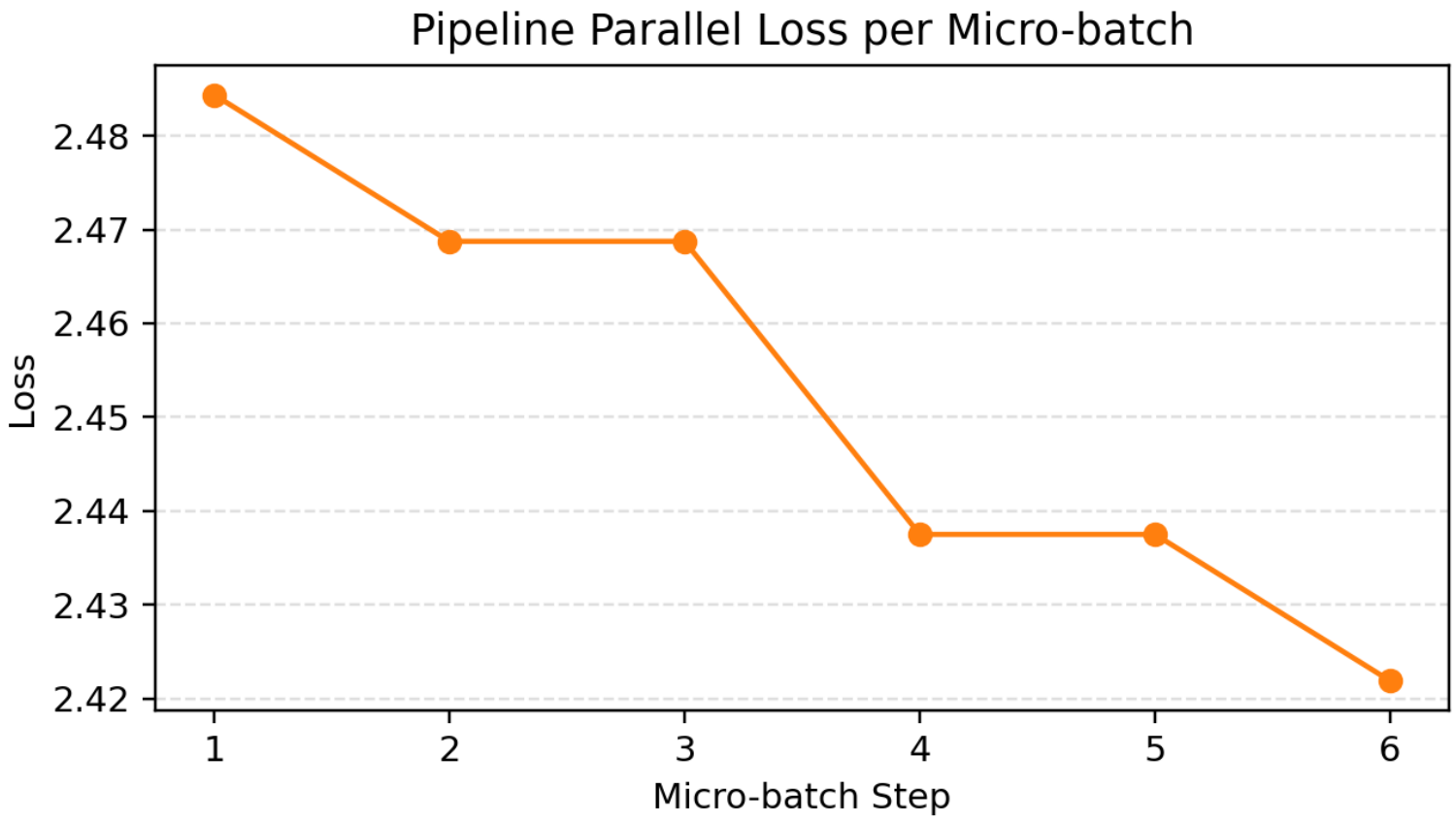
Decision, decisions.

- For simplicity and rapid prototyping, I might start with Hugging Face Accelerate.
- For training massive models with minimal code changes in PyTorch, use DeepSpeed. [this is what I used]
- For a native PyTorch solution with excellent memory efficiency, use FSDP. [am trying this now]
- For achieving peak performance on NVIDIA hardware for LLMs, look to Megatron-LM. [didn’t seem to work well]
- For maximum flexibility, custom parallelism, or TPU training, learn JAX. [seem very complicated]

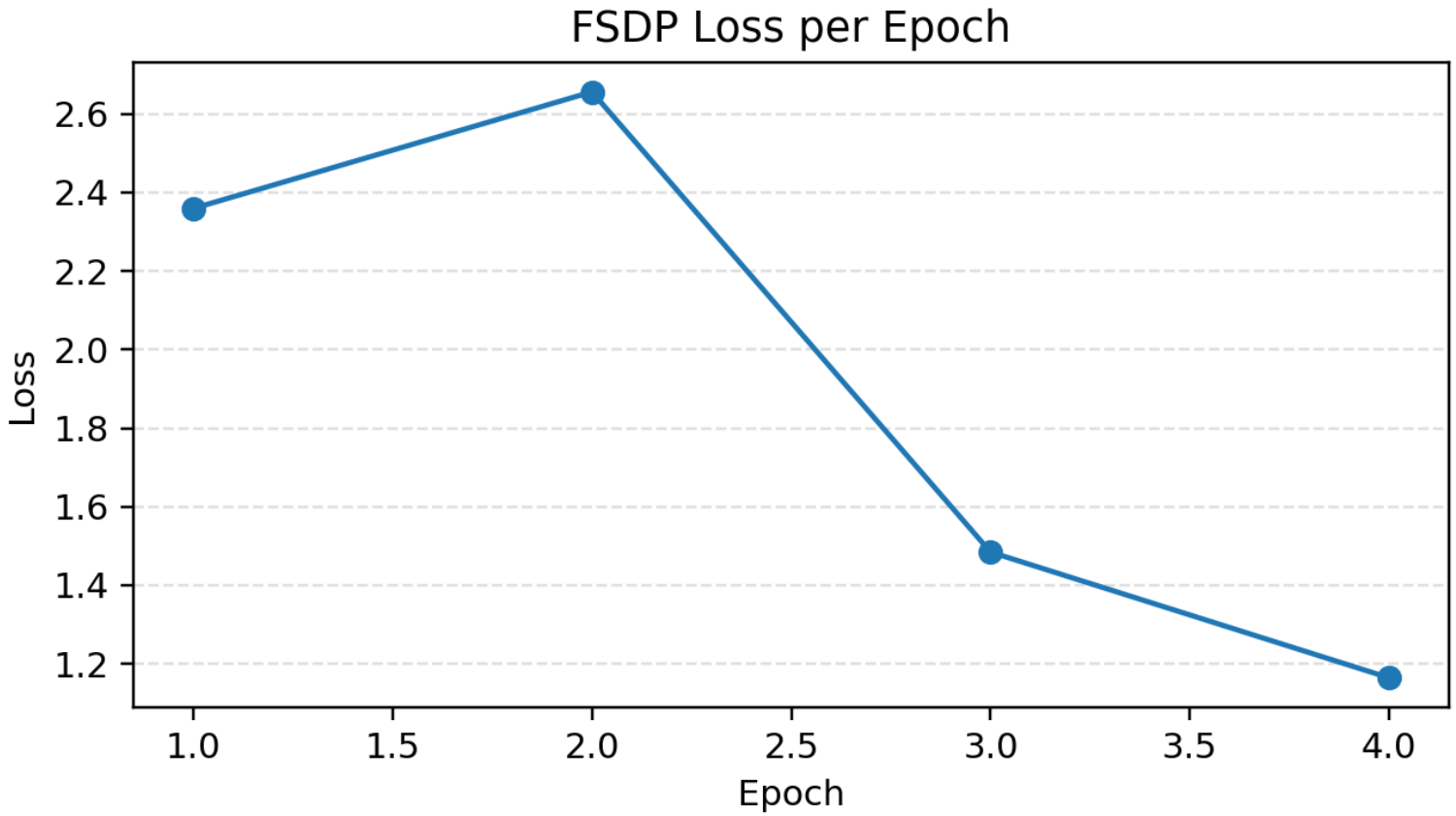
Results - pytorch



Results - pytorch



Results - pytorch




Questions